

pfSense FIREWALL DEPLOYMENT

AND

WAZUH SIEM INTEGRATION

Firewall Rule Enforcement | Syslog Forwarding | Custom Decoder | Custom Detection Rules | Alerting

Prepared by:	Muhammad Qasim
Project Type:	Hands-On Firewall & SIEM Detection-Engineering Lab
Environment:	pfSense CE 2.8.1 Windows 10 Client Wazuh Manager

This project demonstrates deploying pfSense as a segmenting firewall, enforcing an outbound ICMP block policy, forwarding firewall logs to Wazuh over Syslog, and building a custom decoder and hierarchical detection rules to classify and alert on blocked traffic.

1. Project Overview

This project is the third in a series of hands-on SOC labs, building on an earlier standalone Wazuh endpoint-monitoring lab and an Active Directory security-monitoring lab. This time the focus shifted to network-layer security: deploying a firewall to segment and control an endpoint's traffic, then extending Wazuh's detection capability to understand a log format it did not support out of the box.

pfSense Community Edition was deployed as a virtual firewall to place a Windows client behind network segmentation, enforce an outbound ICMP block policy, and forward firewall logs to a centralized Wazuh SIEM. Because Wazuh's built-in pfSense decoder did not correctly parse the RFC 5424 Syslog format used in this environment, a custom decoder and a hierarchical set of detection rules were developed to properly classify and alert on blocked traffic.

This project covers:

- Deployment of pfSense with three segmented interfaces (WAN, LAN, OPT1)
- Creation of a logged firewall rule to block outbound ICMP from the protected LAN
- Validation of the block policy from a Windows client
- Remote Syslog forwarding from pfSense to Wazuh over UDP 514 (RFC 5424)
- Development of a custom Wazuh decoder for the pfSense filterlog format
- Creation of hierarchical custom detection rules with escalating alert levels
- End-to-end validation using Wazuh Logtest and live dashboard alerts

Tools & Environment

Firewall	pfSense Community Edition 2.8.1 (Oracle VirtualBox)
WAN (em0)	10.0.2.15/24 — VirtualBox NAT, external connectivity
LAN (em1)	10.10.10.1/24 — protected internal network
OPT1 (em2)	192.168.56.120/24 — pfSense-to-Wazuh monitoring link
Windows Client	10.10.10.100 (gateway: 10.10.10.1)
Wazuh Manager	192.168.56.118 — Syslog listener on UDP/514
Log Format	Syslog / RFC 5424 (filterlog)

2. pfSense Deployment & Interface Configuration

pfSense Community Edition 2.8.1 was deployed as a virtual machine with three network interfaces, each serving a distinct purpose: WAN (em0) for external connectivity via VirtualBox NAT, LAN (em1) as the protected internal network, and OPT1 (em2) as a dedicated link to the Wazuh Manager. Separating OPT1 from the LAN kept firewall-to-SIEM monitoring traffic isolated from protected client traffic.

```

FreeBSD/amd64 (pfSense.home.arpa) (ttyv1)

login: admin
Password:
VirtualBox Virtual Machine - Netgate Device ID: 8a4b7b4ced09ef73e43f

*** Welcome to pfSense 2.8.1-RELEASE (amd64) on pfSense ***

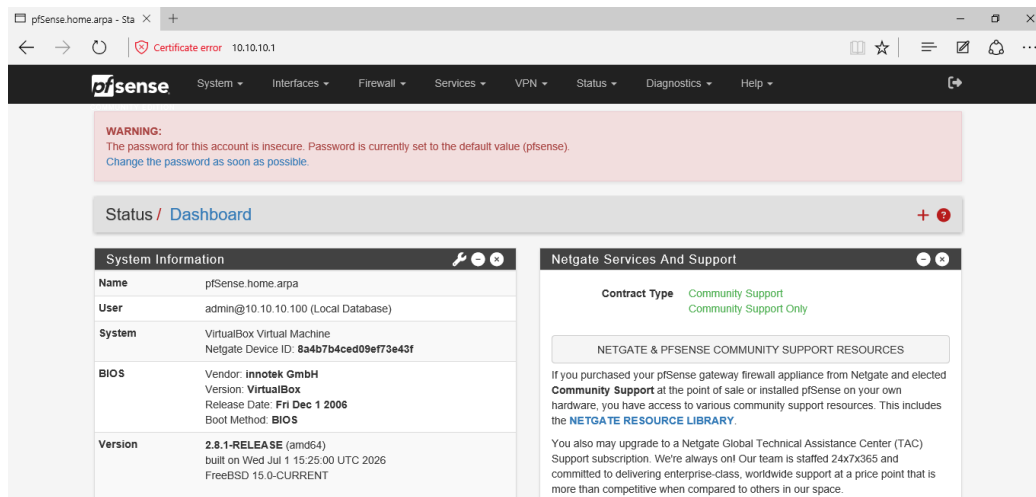
WAN (wan)  -> em0 -> v4/DHCP4: 10.0.2.15/24
              v6/DHCP6: fd17:625c:f037:2:a00:27ff:fead:41c5/64
LAN (lan)   -> em1 -> v4: 10.10.10.1/24
OPT1 (opt1) -> em2 -> v4: 192.168.56.120/24

0) Logout / Disconnect SSH          9) pfTop
1) Assign Interfaces                10) Filter Logs
2) Set interface(s) IP address      11) Restart GUI
3) Reset admin account and password 12) PHP shell + pfSense tools
4) Reset to factory defaults        13) Update from console
5) Reboot system                   14) Enable Secure Shell (ssh)
6) Halt system                     15) Restore recent configuration
7) Ping host                       16) Restart PHP-FPM
8) Shell

```

pfSense console showing the three configured interfaces: WAN (10.0.2.15/24), LAN (10.10.10.1/24), and OPT1 (192.168.56.120/24).

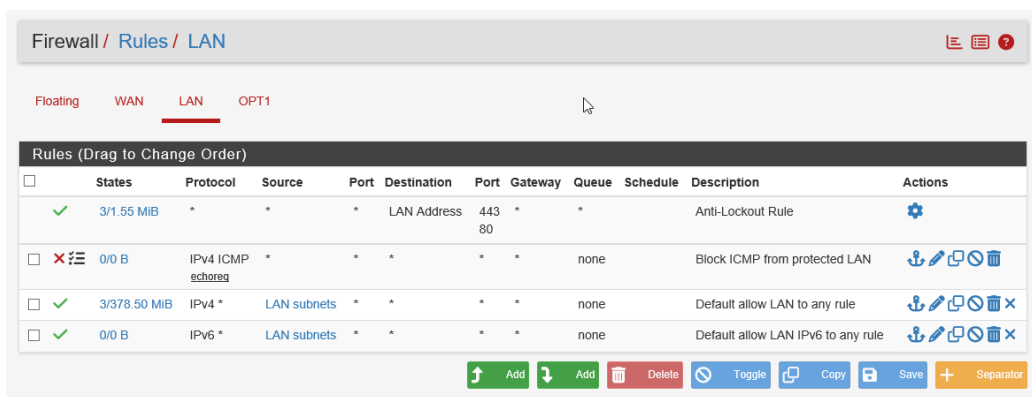
Once the interfaces were assigned, the pfSense web GUI was accessed from the protected Windows client at 10.10.10.100, confirming both LAN connectivity and the deployed pfSense version (2.8.1-RELEASE).



pfSense web dashboard accessed from the Windows LAN client, confirming System Information and version 2.8.1-RELEASE.

3. Firewall Rule: Blocking Outbound ICMP

A firewall rule was created on the pfSense LAN interface to block outbound IPv4 ICMP Echo Request traffic originating from the protected LAN. Logging was enabled on the rule so that every matched request would generate a firewall event. The block rule was placed above the default LAN allow rules, since pfSense evaluates rules top-to-bottom and applies the first match.



pfSense LAN firewall rules: the logged 'Block ICMP from protected LAN' rule placed above the default allow rules.

4. Validation: ICMP Block Test

After the firewall rule was created and saved, the policy was validated by sending ping requests from the Windows client (10.10.10.100) to 8.8.8.8. All four requests timed out, confirming that pfSense was actively blocking outbound ICMP Echo Requests while leaving other permitted traffic unaffected, since the rule targeted ICMP specifically rather than blocking all outbound communication.

```

C:\Windows\system32>ping 8.8.8.8

Pinging 8.8.8.8 with 32 bytes of data:
Request timed out.
Request timed out.
Request timed out.
Request timed out.

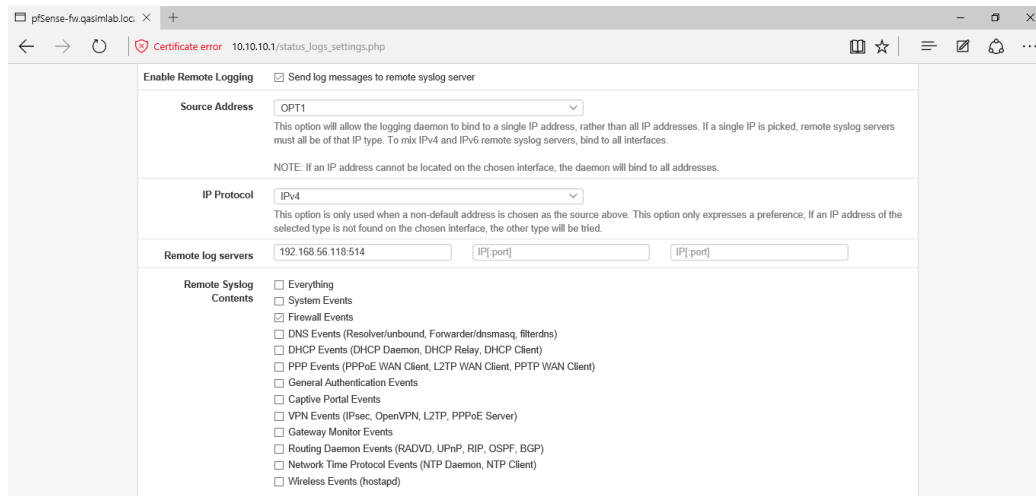
Ping statistics for 8.8.8.8:
    Packets: Sent = 4, Received = 0, Lost = 4 (100% loss),
C:\Windows\system32>

```

Ping test from the Windows client to 8.8.8.8 performed after the ICMP block rule was created — all requests timed out (100% loss).

5. Remote Syslog Forwarding to Wazuh

pfSense was configured to forward firewall events to the Wazuh Manager (192.168.56.118) using Syslog over UDP port 514. The OPT1 address (192.168.56.120) was set as the source address so that monitoring traffic stayed separate from the protected LAN, and only Firewall Events were selected for forwarding.



pfSense Remote Logging configuration: OPT1 source address, remote log server 192.168.56.118:514, Firewall Events selected.

The forwarded messages used the RFC 5424 Syslog format, which defines each message's priority, version, timestamp, hostname, application name, process ID, and the firewall event data itself. Packet capture confirmed pfSense was transmitting filterlog events containing the source (10.10.10.100), destination (8.8.8.8), protocol (ICMP), action (block), and interface (em1). However, Wazuh's built-in pfSense decoder did not correctly parse this RFC 5424 structure, so the logs arrived but were not converted into structured, alertable fields.

6. Custom Wazuh Decoder Development

To resolve the parsing gap, a custom decoder was built using a parent/child structure. The parent decoder, `pfSense-rfc5424`, used a prematch pattern to identify RFC 5424 messages generated by the pfSense filterlog process, ensuring only genuine pfSense firewall logs were processed by the child decoder. The child decoder then applied a regular expression to the comma-separated filterlog fields and mapped them to structured Wazuh fields.

```
<decoder name="pfSense-rfc5424">
  <prematch type="pcr2">filterlog\s+\d+\s+\s+\s+\s+</prematch>
</decoder>

<decoder name="pfSense-rfc5424-ipv4">
  <parent>pfSense-rfc5424</parent>
  <regex type="pcr2" offset="after_parent">^(d+),,,(d+),([^,]+),([^,]+),([^,]+),([^,]+),([d+])</regex>
  <order>pfSense.ruleNum,pfSense.tracker,pfSense.interface,pfSense.reason,pfSense.action,pfSense.direction,pfSense.ipVersion</order>
</decoder>
```

Custom decoder definition (`local_decoder.xml`): `pfSense-rfc5424` parent decoder and `pfSense-rfc5424-ipv4` child decoder with field extraction order.

The decoder extracted the following structured fields from each raw filterlog event:

Decoded Field	Example Value
<code>pfSense.ruleNum</code>	97
<code>pfSense.tracker</code>	1786600176
<code>pfSense.interface</code>	em1

Decoded Field	Example Value
<code>pfsense.reason</code>	match
<code>pfsense.action</code>	block
<code>pfsense.direction</code>	in
<code>pfsense.ipversion</code>	4

This decoding step was important because, although Wazuh can match patterns directly within raw logs, decoding converts important values into structured fields, enabling precise field-based rules, clearer alerts, and efficient dashboard searches.

7. Hierarchical Custom Detection Rules

With the decoder in place, a three-level hierarchical ruleset was created so that detection became progressively more specific: first identifying the event as a pfSense firewall log, then narrowing to blocked traffic, then narrowing further to blocked ICMP traffic specifically.

```
<group name="pfSense,firewall,network_security,">
  <rule id="110100" level="0">
    <decoded_as>pfSense-rcf5424</decoded_as>
    <description>pfSense firewall event</description>
  </rule>
  <rule id="110101" level="5">
    <if_sid>110100</if_sid>
    <field name="pfSense.action">^block$</field>
    <description>pfSense blocked traffic on $(pfSense.interface)</description>
    <group>firewall_block,</group>
  </rule>
  <rule id="110102" level="7">
    <if_sid>110101</if_sid>
    <match>,icmp,</match>
    <description>pfSense blocked ICMP traffic on $(pfSense.interface)</description>
    <group>firewall_block,icmp_block,</group>
  </rule>
</group>
```

Custom ruleset (*local_rules.xml*): rules 110100 → 110101 → 110102 forming a parent/child detection hierarchy.

Rule ID	Parent	Condition	Level	Groups
110100	—	decoded_as = pfsense-rcf5424	0	pfSense, firewall, network_security
110101	110100	pfSense.action = block	5	firewall_block
110102	110101	action = block AND protocol = ICMP	7	firewall_block, icmp_block

The classification logic worked in three escalating stages:

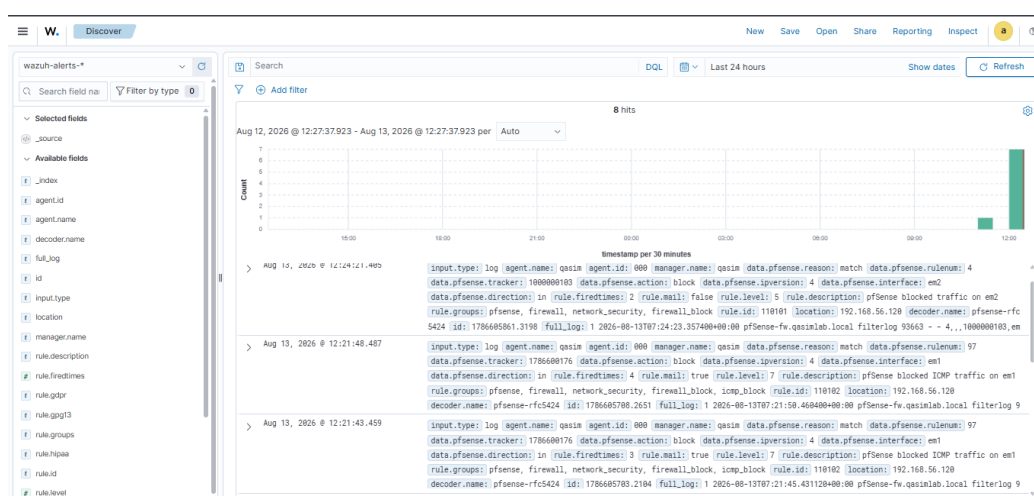
1. Rule 110100 identified the event as a pfSense firewall log (base grouping rule, level 0 — no alert on its own).
2. Rule 110101 checked the decoded `pfSense.action` field for "block" and classified the event as blocked firewall traffic (level 5, `firewall_block` group).

3. Rule 110102 checked for the ICMP protocol within an already-blocked event and classified it as blocked ICMP traffic specifically (level 7, firewall_block + icmp_block groups).

The decoder and rules were tested using Wazuh Logtest before going live, which confirmed the RFC 5424 message matched the custom decoder, the fields were extracted correctly, and rule 110102 fired at alert level 7 as expected.

8. Live Detection: Wazuh Dashboard Alerts

New ping traffic was generated from the protected Windows client to re-trigger the block rule, and the resulting events were confirmed live in the Wazuh dashboard. Each alert showed the custom decoder name (pfsense-rfc5424), the pfSense interface (em1), the firewall action (block), rule ID 110102, alert level 7, and the description "pfSense blocked ICMP traffic on em1."



Wazuh Discover view: live level-7 alerts (rule 110102) showing pfSense blocked ICMP traffic on interface em1, decoded via pfsense-rfc5424.

The complete monitoring workflow validated by this project was:

```
Protected Windows client traffic
-> pfSense firewall inspection (LAN rule)
-> ICMP traffic blocked and logged
-> RFC 5424 Syslog message sent over UDP 514 (OPT1 -> Wazuh Manager)
-> Custom Wazuh decoder (pfsense-rfc5424) extracts structured fields
-> Hierarchical rules 110100 -> 110101 -> 110102 classify the event
-> Level-7 alert displayed in the Wazuh Dashboard
```

9. Conclusion

This project extended the earlier Wazuh SIEM work into network-layer detection engineering. A segmented pfSense firewall was deployed, a logged ICMP block policy was enforced and validated, and firewall events were

forwarded to Wazuh over Syslog. When the default decoder proved insufficient for the RFC 5424 format in use, a custom parent/child decoder and a three-level hierarchical ruleset were built, tested with Logtest, and validated end-to-end with live alerts in the Wazuh dashboard.

Together with the earlier endpoint-focused SIEM lab and the Active Directory security-monitoring lab, this project rounds out a practical SOC skill set spanning endpoint monitoring, identity-based attack detection, and network firewall log integration — including the detection-engineering work of writing custom decoders and rules when out-of-the-box support falls short.